Local GPU Training Run Log

Companion to `docs/08_CLOUD_TRAINING_RUN_LOG.md`. That log covers the CPU-only proof-of-pipeline run done in the cloud build sandbox. This one covers the real training run on the target machine, which is what the final report should cite.

Date: 2026-08-24 **Machine:** Windows 11, NVIDIA GeForce RTX 4060 Laptop GPU (8 GB), driver 592.82 / CUDA 13.1 **Environment:** Python 3.11.9, torch 2.5.1+cu121, ultralytics 8.4.127, `torch.cuda.is_available() == True`

1. What changed vs. the cloud run

The cloud run was CPU-only, 4 epochs, and could not download ImageNet weights, so it trained from random initialisation. Every number in it was a pipeline-correctness check, not a result. This run uses the GPU, the full epoch budget from `configs/config.yaml`, and real ImageNet-pretrained initialisation.

The cloud-run artifacts were not deleted — they are preserved under `outputs/cloud_cpu_run_archive/` for comparison.

2. Dataset audits reproduced locally

All three audits reproduce the documented numbers exactly, against the local copies extracted from the original zips:

Dataset	Result	Matches docs/03?
A (Corn_3_Classes)	1,050 images, 3 classes balanced 350/350/350, 0 corrupt, 4 duplicate pairs	yes
B (MaizeData)	17,724 images, Bhihilifa 6,480 / SanzalSima 5,100 / WangDataa 6,144, 0 corrupt, 11 duplicate pairs	yes
C (detection)	42,602 instances over 1,251 images, 1 invalid box	yes

Post-dedup manifest sizes: Dataset A 1,046 (736 train / 155 val / 155 test), Dataset B 17,713 (12,403 train / 2,655 val / 2,655 test). MD5 dedup runs before the split, so no duplicate can straddle train and test.

Synthetic defect generation produced 4,200 manifest rows, perfectly balanced across `healthy` / `cracked` / `discolored_mold` / `insect_damaged` (1,050 each).

3. Bugs this run exposed

The cloud environment could not have caught 3.1–3.3, 3.6 or 3.7 — they are Windows-, GPU-, scheduling-, or whole-system-specific. All seven are fixed.

3.1 Dataset classes were unpicklable on Windows (blocker)

`src/data/datasets.py` stored the PIL `Image` **module** as an instance attribute (`self._Image = Image`). Linux DataLoader workers use `fork` and inherit memory, so this never mattered there. Windows uses `spawn`, which pickles the dataset object to send it to each worker — and module objects cannot be pickled. Every training script crashed instantly with `TypeError: cannot pickle 'module' object`. **Fix:** import PIL at module scope instead of storing it per-instance.

3.2 GPU was idle at ~2% — training was data-starved (18x slowdown)

First working run took ~30 s/epoch on 736 images while `nvidia-smi` showed 2% GPU utilisation. The DataLoaders used `num_workers=4` with default `persistent_workers=False`, so on Windows all four worker *processes* were spawned and torn down every single epoch. Worker startup cost far exceeded the actual compute. **Fix:** `persistent_workers=True` and `pin_memory=True` on every DataLoader in the three training scripts. **30 s/epoch → 1.7 s/epoch.**

This is why the Dataset B budget in the earlier status report (a projected ~14 hours) was wrong — it extrapolated from the broken configuration. Both datasets now use the identical full epoch budget from `configs/config.yaml`, which also makes the A-vs-B ablation comparison a fair one.

3.3 YOLO weights were written where the pipeline could never find them

`train_detection.py` passed `project=outputs/checkpoints` to Ultralytics. Ultralytics resolves a **relative** `project` against its own `runs/` directory, so the weights landed in `runs/detect/outputs/checkpoints/detection_corn/weights/best.pt` while `unified_pipeline._get_detection_model()` looked in `outputs/checkpoints/detection_corn/weights/best.pt`. Detection would have reported "model not available" forever despite having trained successfully. **Fix:** pass an absolute path for `project`.

3.4 The configured Gemini model had been retired

`gemini-2.0-flash` returned `404 ... no longer available to new users`. The API key itself was valid. **Fix:** `gemini-3.6-flash` (the replacement the API itself names), in both `backend/config.py` and `configs/config.yaml`.

3.5 Similarity lookup hardcoded the experiment name

`unified_pipeline`'s embedding/similarity lookup hardcoded `experiment="full"`, so it broke whenever only a different variant had been trained — and would have silently mislabelled which checkpoint served a result. **Fix:** `_resolve_experiment()` with a fallback chain (`full → contrastive_only → attention_only → baseline`, whichever exists on disk), reporting both the requested and the actually-used experiment in the API response.

3.6 A stage was killed launching immediately after the previous one (exit 127)

`variety_b_baseline` died 30 s after starting, with **exit 127 and no Python traceback** — its log contained a single line (`Device: cuda | dataset=b | experiment=baseline`), meaning the process was killed rather than raising. It had launched *zero seconds* after `contrastive_b` finished and began releasing ~6.8 GB of VRAM, so it attempted CUDA initialisation against a GPU still tearing down the previous context. Re-

running the exact same command by hand succeeded immediately, confirming the failure was transient rather than a code defect.

This mattered more than a one-off crash: the orchestrator exits on stage failure, so the chain had stopped with four stages left and would have sat idle indefinitely. **Fix:** a `settle()` guard in the orchestrator that polls `nvidia-smi` and refuses to start a stage until VRAM drops below 1.5 GB (120 s cap, then a 5 s grace), plus a one-shot retry before giving up. The killed run also left an orphaned `variety_b_baseline_best.pt` with no metrics file and no `.done` marker — a half-trained checkpoint that could later have been mistaken for a real one — which was deleted before restarting. After the fix the remaining five stages ran back-to-back with 5–6 s gaps and no further failures.

3.7 The served pipeline scored 22 points below the reported accuracy

Found by running the finished application against held-out data, not by any test. `train_variety.py` trains and evaluates on **whole images**; `unified_pipeline` serves the classifier **YOLO crops**. Those are different input distributions, and Dataset A's images are single-seed close-ups where the surrounding context is part of what the model learned. On the same 155-image held-out split: whole image **100.00%**, served crop path **77.63%** — 34 disagreements, with crop predictions skewed to `Rugosa` 79 / `Indurata` 22 against a true distribution of ~52/51/52, and wrong predictions clustered at near-chance confidence (0.44–0.65).

The test suite could not catch this: every stage was exercised against data matching its own training distribution, so the mismatch only appeared once the stages were chained.

Fix: sweeping the crop context margin showed 0% → 77.63%, 15% → 98.03%, **30% → 100.00%** (50/75/100% likewise 100%). 30% is the smallest margin that fully recovers accuracy; larger ones risk pulling neighbouring kernels into the crop on dense images. Added as `detection.crop_context_pad: 0.30` and applied in `AnalysisPipeline.crop_seed()` with bounds clamping. After the fix: served path **100.00%**, **0** disagreements, distribution 50/51/51. Measurable any time via `python -m src.analysis.eval_serving_gap --dataset a`.

The structural fix — retraining the classifier on crops so the distributions match by construction rather than by a tuned margin — is still the more robust option and is noted as residual work in `docs/09_PROJECT_STATUS_REPORT.md` §6.6.

4. Training results

4.1 Contrastive pretraining (SimCLR, NT-Xent)

Dataset	Images	Epochs	Loss (first → last)
A	1,050	60	3.968 → 2.962
B	17,713	60	3.200 → 2.885

Dataset B's loss was effectively converged by epoch ~25 (2.900) and moved only 0.015 over the remaining 35 epochs. It did **not** collapse — with `pretrain_batch_size: 64` the NT-Xent collapse floor is $\ln(2N-1) = \ln(127) = 4.84$, and the run finished well below it at 2.885, so the encoder learned real structure. Epoch

time degraded from 63 s to ~257 s over the run as GPU contention increased; the stage took 3 h 22 m wall-clock.

4.2 Variety classification — Dataset A (test split, 155 images)

Experiment	Test accuracy	F1 (macro)
baseline	99.35%	0.9936
attention_only	100.00%	1.0000
contrastive_only	100.00%	1.0000
full (proposed)	100.00%	1.0000

Every variant early-stopped within 8–12 epochs. **Dataset A is saturated**: with ImageNet initialisation the task is solved by epoch 2, and three of the four variants reach a perfect test score. The 0.0064 macro-F1 gap between baseline and the rest is a single misclassified image out of 155 — well inside run-to-run noise. This dataset therefore **cannot** discriminate between the architectures, which is consistent with the literature survey (`docs/02`, row 1: prior published work reports 99–100% on this exact dataset). Do not claim the attention/contrastive modules are validated by Dataset A. See `docs/10_RESULTS_COMPARISON.md`.

4.3 Variety classification — Dataset B (test split, 2,655 images)

Experiment	Test accuracy	F1 (macro)	Errors / 2,655	Δ macro-F1 vs baseline
baseline	99.89%	0.9988	3	—
attention_only	99.89%	0.9988	3	+0.0000
contrastive_only	**99.96%**	**0.9996**	**1**	+0.0008
full (proposed)	99.92%	0.9992	2	+0.0004

Dataset B was the more informative of the two for the ablation, being ~17x larger — and it returned the same verdict as Dataset A. **`attention_only` is indistinguishable from `baseline`** (identical macro-F1 to four decimals, same 3 errors), and **the proposed `full` configuration is beaten by `contrastive_only`** (2 errors vs 1) — adding attention on top of contrastive pretraining made the model marginally worse. The total spread across all four variants is 0.0008 macro-F1, i.e. **two images out of 2,655**, which is well inside single-run seed variance.

Both variety datasets are therefore saturated and neither can discriminate between the architectures. The defensible conclusion for the report is that **on this data, cognitive attention and contrastive pretraining produce no measurable benefit over a plain EfficientNet-B0 with ImageNet initialisation** — a valid negative result, given the baseline itself reaches 99.89%. Do not present `full` as the best variant; on Dataset B it is not. See `docs/09_PROJECT_STATUS_REPORT.md` §6.7 for what a proper test would require.

4.4 Synthetic defect classifier (4 classes, 630-image test split)

Test accuracy **99.68%**, macro-F1 0.9968. Per class:

Class	Precision	Recall	F1	Support
healthy	0.988	1.000	0.994	158
cracked	1.000	0.987	0.994	158
discolored_mold	1.000	1.000	1.000	157
insect_damaged	1.000	1.000	1.000	157

Only 2 errors, both thin-crack images read as healthy.

This number must be reported with its caveat. ~99.7% here is the *expected* result and is not evidence of real defect-detection capability: the model is being asked to recognise procedurally generated patterns drawn by a known OpenCV routine, so the classes are far more visually separable than real damage would be. It demonstrates that the architecture and serving path can learn and serve a defect task. It says nothing about real-world plant pathology. See `docs/06_SYNTHETIC_DEFECT_POLICY.md`.

4.5 Seed detection (YOLOv8n, Dataset C, 80 epochs)

Metric	Value
mAP@50	0.9822
mAP@50-95	0.6614
Precision	0.9791
Recall	0.9692

Strong localisation (mAP@50 98.2%) with looser box-tightness at high IoU (mAP@50-95 66.1%) — expected for small, densely packed, touching objects averaging ~34 seeds per image.

5. End-to-end verification

`pytest tests/` — **29 passed, 0 skipped** once every checkpoint existed (16 pre-existing tests + 13 new integration tests in `tests/test_integration_e2e.py`, which exercise the real trained models rather than the graceful-degradation paths).

Live API check on a held-out real Indurata photograph: - 1 seed detected, variety `Zea_mays_Indurata` @ 95.25% (correct), served by `variety_a_full` - synthetic defect `healthy` @ 97.63%, `is_synthetic_model: true` - similarity search returned three genuine Indurata neighbours, nearest-first - `warnings: []` — no stage degraded

Live frontend check (Vite dev server against the running backend), all 6 pages: - **Dashboard** — API online, Gemini configured - **Analyze** — real upload → 2 seeds, both `Zea_mays_Rugosa` @ 98.6% (correct), synthetic disclaimer rendered inline per seed - **Batch** — 3 images, 3 succeeded / 0 failed, 6 seeds, variety distribution matched the source folders exactly, avg confidence 0.965 - **History** — all analyses persisted and retrievable - **Copilot** — Gemini answered from stored results only - **System Info** — supported vs. explicitly-not-supported lists render

Gemini guardrail check: asked "can you confirm this seed is free of disease?" against a real analysis. It correctly refused — stated the system cannot diagnose disease and that the defect output is a simulated

prediction from a synthetic model. The `LIMITATION_CONTEXT` system instruction is doing its job.

6. Honest summary for the report

- Seed **detection** and **variety classification** are real, trained on real labelled data, and perform well.
- **The ablation returned a null result on both datasets.** Dataset A and Dataset B are each saturated (≥99.35% and ≥99.89% from a plain ImageNet-initialised baseline), so neither can discriminate between the architectures. On Dataset B, `attention_only` is identical to `baseline` to four decimal places, and the proposed `full` variant is beaten by `contrastive_only`. **Do not claim that cognitive attention or contrastive pretraining improves accuracy in this project — the evidence does not support it.** Report the spread (0.0008 macro-F1 on B; two images out of 2,655) and state the conclusion as a negative result. This is a finding about the datasets, not a defect in the implementation.
- The **defect classifier is a demonstration on synthetic data** and its high accuracy is a property of the synthetic generator, not evidence of pathology detection.
- **Similarity search** is visual/feature similarity, not certification.
- **Grad-CAM** is an explainability heatmap, not segmentation and not defect area.

---

7. Second run — August 25–26, 2026

The first run trained everything. This run established that a substantial part of it had been measuring the wrong thing, and rebuilt those parts.

7.1 Timeline

Stage	Duration	Outcome
Dataset 4 download + audit	~12 min	4,846 images, 0 corrupt, 0 overlap with A/B/C
Group-aware re-split of Dataset B	<1 min	127 sources → 89/19/19, zero cross-split contamination
Contrastive pretrain, B train split only	45.6 min	loss 3.256 → 2.887, 60 epochs
`baseline` (grouped)	10.1 min	85.35%
`attention_only` (grouped)	5.8 min	85.31%
`contrastive_only` (grouped)	4.9 min	89.06%
`full` (grouped)	7.1 min	87.97%
Contrastive pretrain, pooled train splits	~59 min	16,730 images, loss 3.184 → 2.887
Unified two-head model	~17 min	early stop at epoch 30/40
Quality distribution reference	~8 min	3,401 embeddings, threshold 0.2056

Total GPU time ≈ 2h40m across two sessions.

7.2 Bugs and defects this run exposed

1. Dataset B split leakage — invalidated the project's headline result. All 127 source seeds had augmented copies in all three splits; 2,655 of 2,655 test images had a same-seed sibling in training. Detected by parsing the provenance encoded in the filenames while building a combined manifest. Full analysis in `09_PROJECT_STATUS_REPORT.md` §6.9. Fixed by `src/data/group_split.py`; the invariant is now asserted by `tests/test_unified_model.py::test_no_group_spans_more_than_one_split`.

2. Contrastive pretraining saw the test set. `pretrain_contrastive.py` called `list_all_filepaths(data_root)`, enumerating every image regardless of split. The original `contrastive_encoder_b.pt` had therefore seen all 2,655 test images before they were ever evaluated, compromising the `contrastive_only` and `full` arms by a route entirely independent of bug 1. Fixed with `--manifest`, which restricts to training rows; the old code path now emits a warning naming the risk.

3. A measurement error of my own, caught before it caused harm. An initial check reported that 83.2% of Dataset A's test images had a near-duplicate in training, which would have implied Dataset A was leaky too and required four more retraining runs. It was wrong: a 64-bit perceptual hash lacks the resolution to separate distinct seeds in a homogeneous dataset. Re-measured against the dataset's own similarity distribution (mean pairwise 0.697, p95 0.925), the 0.95 threshold that produced the figure selects the top 5% of *all* pairs. At a genuine duplicate bar (≥0.99), **zero** Dataset A test images have a training twin. Dataset A is clean; its four ablations did not need retraining, and were not retrained. Recorded here because the retraction changed the scope of the work and is exactly the kind of near-miss worth documenting.

4. Metrics reported a sample size that did not exist. Image-level accuracy over Dataset B's 2,669 test images treats ~140 augmented views of one kernel as 140 independent observations. `train_variety.py` now also reports group-level accuracy with `n_groups` as the honest n, and significance testing operates on per-seed values rather than per-image ones.

5. `compare_experiments.py` exited non-zero after succeeding. The generated tables contain `Δ` and `–`, which cp1252 — the default Windows console encoding — cannot encode. The file was written correctly and then the terminal echo raised `UnicodeEncodeError`, so the script reported failure on a successful run. Now guarded.

6. The quality head extrapolates confidently. Not a code defect but a model property, and the most important one for the platform's honesty: applied to Dataset A the head calls 71% of a clean variety dataset defective at 89% mean confidence, and softmax confidence does not distinguish this from competence — it is *higher* on some out-of-distribution imagery than on real test data. Addressed by the distribution gate (`src/analysis/build_quality_reference.py`), which catches 100% of Dataset A at a 6.2% in-distribution false-flag rate.

7.3 What the numbers became

```
experiment          ORIGINAL (leaked)  GROUPED (honest)     drop
baseline                        99.89%            85.35%   14.54pp
attention_only                  99.89%            85.31%   14.57pp
contrastive_only                99.96%            89.06%   10.90pp
full                            99.92%            87.97%   11.95pp
```

Paired Wilcoxon over 19 source seeds, Holm-corrected for six comparisons: `full` beats `attention_only` by 2.63pp on 12 of 19 seeds, p = 0.0052 — the one comparison surviving correction. Attention alone remains indistinguishable from baseline (p = 0.68).

The project's central hypothesis moved from "not supported" to "supported, with stated limitations." The original negative result was not a finding about the architecture; it was a finding about a split that left no headroom in which a difference could appear.

7.4 Honest summary

The first run produced a complete, working, thoroughly documented system whose headline experimental claim rested on an invalid split. The second run found that, corrected it, re-ran the affected experiments, added the project's first real quality capability from expert-labelled data, consolidated three served models into one, and built a gate so the new capability states when it is guessing.

The result that matters is not that accuracy went up. It went **down**, by fourteen points, and the drop is the point: the earlier figure was not real. What went up is the amount of the system's output that can be defended under questioning.